

CogNet-1B

A Non-Transformer Language Model with Cognitive Routing and Hierarchical Memory

Official Architecture Whitepaper

Version 2.0 — June 2025

AFKmoney

Abstract

CogNet-1B is a ~1.06 billion parameter language model built on a novel non-transformer architecture. Unlike conventional transformer models that rely on self-attention mechanisms with $O(n^2)$ per-layer complexity, CogNet employs cognitive routing with vectorized channel processing and a three-tier hierarchical memory system (working, episodic, and semantic), achieving $O(n)$ per-layer computational complexity. This whitepaper presents the architecture, its key innovations, the optimized training pipeline, and the empirical benchmarking methodology used to validate performance claims on real hardware.

Keywords: non-transformer LLM, cognitive routing, hierarchical memory, $O(n)$ complexity, channel processing, Flash Attention, vectorized computation

1. Introduction

The dominant paradigm in large language model architecture has been the transformer, originally introduced by Vaswani et al. in 2017. While transformers have achieved remarkable success across natural language processing, computer vision, and multimodal tasks, their core self-attention mechanism exhibits quadratic computational complexity $O(n^2)$ with respect to sequence length n . This fundamental scaling limitation constrains the maximum context window size, increases training costs, and limits deployment on memory-constrained hardware.

CogNet-1B proposes a fundamentally different approach. Rather than relying on global self-attention over all token pairs, CogNet routes input through parallel cognitive channels and maintains information across sequence positions using a hierarchical memory system with three distinct tiers: working memory for immediate context, episodic memory for medium-range patterns, and semantic memory for long-range abstractions. Each memory tier is read using scaled dot-product attention (SDPA) with Flash Attention kernels, but the attention is applied only to a fixed-size set of memory slots rather than all sequence positions, resulting in $O(n)$ per-layer complexity.

This whitepaper details the CogNet-1B architecture, its key innovations over both traditional transformers and the original CogNet implementation, the complete optimized training pipeline, and the empirical benchmarking methodology that ensures all performance claims are measured on real hardware rather than estimated.

2. Architecture

2.1 Model Specifications

Parameter	Value	Description
Hidden dim	2048	Main representation dimension
Blocks	16	Number of CogNet blocks
Channels per block	8	Parallel cognitive channels
Channel dim	384	Per-channel representation size
FF dim	8192	Feed-forward dimension (Fused SwiGLU)
Working memory slots	128	Short-term context storage
Episodic memory slots	256	Medium-range pattern storage
Semantic memory slots	512	Long-range abstraction storage
Vocabulary size	136	Character-level tokenizer
Max sequence length	512	Training sequence length
Total parameters	~1.06B	Approximately 1.06 billion

2.2 CogNet Block

Each CogNet block processes input through a sequence of operations that differ fundamentally from a standard transformer block. While a transformer block applies layer normalization, multi-head self-attention, residual connection, layer normalization, feed-forward network, and another residual connection, the CogNet block replaces self-attention with cognitive routing and hierarchical memory reads.

The processing pipeline within each CogNet block is as follows. First, RMSNorm is applied to the input. Then the cognitive router splits the normalized input into parallel channels, where each channel specializes in different aspects of the representation. Next, the three memory tiers (working, episodic, semantic) are read in parallel using SDPA with Flash Attention, producing gated contributions from each tier. These contributions are combined and merged back into the hidden dimension. A residual connection adds this output to the original input. Then a second RMSNorm is applied, followed by a Fused SwiGLU feed-forward network with a final residual connection.

2.3 Cognitive Router

The cognitive router is the core mechanism that distinguishes CogNet from transformer architectures. Rather than computing attention weights between all pairs of sequence positions (which yields $O(n^2)$ complexity), the router projects the hidden state into multiple parallel channels. Each channel operates on a lower-dimensional representation (`channel_dim = 384` vs `hidden_dim = 2048`), enabling specialized processing without the quadratic cost of global attention.

In the original CogNet implementation, channels were processed sequentially using a Python for-loop, which introduced significant overhead on GPU due to kernel launch costs and poor parallelism. The optimized implementation uses vectorized channel processing: all 8 channels are processed simultaneously in a single batched operation. The input is reshaped to `(batch, seq_len, num_channels, channel_dim)`, and all channel operations are applied via batched matrix multiplications and element-wise operations. This eliminates the sequential bottleneck and enables full utilization of GPU parallelism.

2.4 Hierarchical Memory System

The hierarchical memory system provides CogNet with its mechanism for maintaining information across sequence positions. It consists of three tiers with increasing capacity and decreasing granularity, inspired by cognitive science models of human memory. Each tier stores fixed-size key-value pairs in memory slots. The contents of these slots are updated at each sequence position and read using scaled dot-product attention with Flash Attention kernels.

Memory Tier	Slots	Key Dim	Purpose	Analogy
Working	128	64	Immediate token-level context	Working memory / attention buffer
Episodic	256	64	Medium-range patterns and phrases	Episodic memory / chunk recall
Semantic	512	64	Long-range abstractions and topics	Semantic memory / knowledge

Each memory tier is read using a separate SDPA call with Flash Attention. The query is derived from the current channel representation, while keys and values come from the memory slots. The three tier outputs are gated independently and then combined. This parallel tier read strategy means that the computational cost scales as $O(n \times \text{slots})$ for each tier, and since the number of slots is a fixed constant independent of sequence length, the overall per-layer complexity is $O(n)$.

2.5 Complexity Analysis

The per-layer computational complexity of CogNet is $O(n)$ in sequence length, compared to $O(n^2)$ for standard transformer self-attention. This linear scaling arises because the most expensive operation in each CogNet block is the memory tier reads, which compute attention over fixed-size slot sets rather than the full sequence. The feed-forward network (Fused SwiGLU) also operates in $O(n)$ since it is applied independently at each position. The cognitive router involves matrix multiplications that are linear in sequence length. Therefore, doubling the sequence length approximately doubles the compute time per layer, rather than quadrupling it as in a transformer.

Operation	Transformer	CogNet	Notes
Self-attention / Memory reads	$O(n^2)$	$O(n \times S \times d)$	S = total memory slots (fixed)
Feed-forward network	$O(nd^2)$	$O(nd \times \text{ff})$	Both linear in n
Cognitive routing	N/A	$O(nd \times \text{ch})$	Unique to CogNet, linear in n
Per-layer total	$O(n^2)$	$O(n(S+d+\text{ff}+\text{ch}))$	Linear vs quadratic in n

Where n = sequence length, d = hidden dimension, S = total memory slots (896), ff = feed-forward dimension, ch = channel dimension.

3. Key Innovations

3.1 Vectorized Channel Processing

The original CogNet implementation processed each of the 8 cognitive channels in a sequential Python for-loop. This approach, while logically clear, introduces severe performance penalties on GPU hardware. Each iteration of the loop requires a separate kernel launch, incurring overhead that dominates the computation time for small channel dimensions. Furthermore, the sequential nature prevents the GPU from exploiting the natural parallelism across channels.

The optimized implementation eliminates the for-loop entirely. All channels are processed simultaneously through batched operations. The input tensor is reshaped from (batch, seq_len, hidden_dim) to (batch, seq_len, num_channels, channel_dim), and all linear projections, activations, and memory interactions are applied via batched matrix multiplications. This transformation reduces kernel launch overhead from 8 separate launches to a single batched operation and enables the GPU scheduler to fully utilize available parallelism.

3.2 Fused SwiGLU Feed-Forward Network

The SwiGLU activation function, introduced by Shazeer (2020), has become a standard replacement for the traditional ReLU in transformer feed-forward networks. It computes $\text{output} = \text{Swish}(xW_{\text{gate}}) \times (xW_{\text{up}})$, where $\text{Swish}(x) = x \times \text{sigmoid}(x)$. In a naive implementation, this requires two separate linear projections (gate and up) followed by element-wise multiplication.

The Fused SwiGLU optimization concatenates the gate and up projection weights into a single weight matrix of shape $(\text{hidden_dim}, 2 \times \text{ff_dim})$. A single matrix multiplication produces both the gate and up activations, which are then split and combined. This reduces the number of GEMM operations from two to one, which is particularly beneficial on GPU where large matrix multiplications are the primary compute bottleneck.

3.3 RMSNorm and RoPE

The original CogNet used LayerNorm for normalization and learned positional embeddings. The optimized implementation replaces both with RMSNorm and Rotary Position Embeddings (RoPE), respectively. RMSNorm computes the root mean square of the input vector and scales accordingly, without the mean-subtraction and bias terms present in LayerNorm. This simplifies the computation and has been shown to provide equivalent or superior training stability while being slightly faster to compute.

RoPE encodes positional information by rotating the query and key vectors in the attention computation, rather than adding learned positional embeddings to the input. This has several advantages: it naturally handles variable sequence lengths at inference time, it provides relative positional information rather than absolute, and it eliminates the need to store and compute a positional embedding table. For CogNet, RoPE is applied within the memory tier attention computations.

3.4 SDPA with Flash Attention for Memory Tiers

The memory tier reads are the most computationally significant operations in each CogNet block. The optimized implementation uses PyTorch's Scaled Dot-Product Attention (SDPA) with Flash Attention kernels for all three memory tier reads. Flash Attention is an IO-aware exact attention algorithm that reduces the number of HBM (high-bandwidth memory) reads and writes from $O(n^2)$ to $O(n)$, achieving wall-clock speedups of 2-4x over standard attention implementations while computing the exact same output numerically.

Each memory tier is read with a separate SDPA call: one for working memory (128 slots), one for episodic memory (256 slots), and one for semantic memory (512 slots). The three calls are independent and could in principle be further parallelized. Each call uses the current channel representation as the query and the memory slot keys and values, producing an output that is then gated before combination.

4. Training Pipeline

4.1 Data Sources (A-B-C-D-E)

The training pipeline integrates data from five distinct sources, each contributing different types of linguistic and structural knowledge. This diverse data mixture ensures that the model encounters a wide range of natural language patterns, code structures, and domain-specific content during training.

Part	Source	Content	Scale Target
A	HuggingFace datasets (7 sources)	wikitext-103, codeparrot-clean, fineweb, b5B, fra, the-stack-smol, alpaca-cleaned, c4-en	5B
B	CogNet HF repo	Pre-tokenized .pt files from thefinalboss/CogNet-1B	Variable
C	AICL GitHub repo	JSONL datasets, .aicl examples, source code, docs, tests (10x repeated)	Full, 400M
D	HF scripts	Python/JSON/MD from CogNet-1B repository (3x weight)	Variable
E	Synthetic data	Code templates + English + French sentences	50M chars

After all sources are tokenized, they are merged into a single tensor and shuffled using a random permutation. The resulting dataset is saved as train_merged.pt for efficient loading during training. The AICL data is repeated 10 times to increase its weight in the training mixture, reflecting the importance of structured AICL-specific knowledge for the target use case.

4.2 Character Tokenizer

CogNet-1B uses a character-level tokenizer with a vocabulary of 136 tokens: 4 special tokens (PAD, UNK, BOS, EOS) plus 132 printable ASCII and French-accented characters. This minimal vocabulary is deliberate: it eliminates the need for a complex subword tokenization pipeline, reduces vocabulary-related parameters in the embedding layer, and ensures that every character in common English and French text has a direct token representation. The trade-off is longer token sequences compared to BPE tokenizers, but this is mitigated by CogNet's O(n) per-layer complexity which scales gracefully with sequence length.

4.3 Training Optimizations

#	Optimization	Benefit
1	BF16 mixed precision	2x throughput vs FP32 on Ampere+ GPUs
2	RMSNorm + RoPE	No learned positional table, simpler computation
3	Vectorized channel processing	No Python for-loops, full GPU parallelism
4	SDPA / Flash Attention for memory tied	Fused attention kernels, reduced HBM I/O
5	Fused SwiGLU	Single matmul for gate+up projections
6	Gradient checkpointing	~3x memory savings, enables larger batches
7	torch.compile()	Kernel fusion, reduced Python overhead
8	FSDP multi-GPU	Near-linear multi-GPU scaling
9	Fused AdamW optimizer	Fused CUDA kernel for optimizer step
10	CUDA prefetch data pipeline	Overlaps Host-to-Device transfer with compute

#	Optimization	Benefit
11	Async checkpointing	Background thread saves, no training pause
12	Sequence length warmup	128 to 512 curriculum during warmup period
13	8-bit optimizer (optional)	50% less VRAM for optimizer states via bitsandbytes

4.4 Real Benchmark Methodology

All performance claims for CogNet-1B are based on empirical measurements, not theoretical estimates. The training script implements a mandatory benchmark phase at the start of every training run. This benchmark consists of two phases: a warmup phase of 3 steps to heat up compilation caches and CUDA memory allocations, followed by a measurement phase of 10 complete training steps (forward pass, backward pass, and optimizer step) timed with CUDA synchronization to ensure accurate measurement.

The benchmark reports steps per second and tokens per second measured on the actual hardware configuration being used, and calculates the estimated time to completion based on the measured speed. This measurement replaces all fabricated or estimated speed claims with real, verifiable data. The benchmark results are saved to a JSON file (benchmark_results.json) for reproducibility and comparison across different hardware configurations. During training, each log line displays an ETA calculated from the measured benchmark speed, providing a stable and accurate estimate rather than a fluctuating instantaneous speed calculation.

4.5 Checkpoint Management

The training pipeline maintains three checkpoint files that are overwritten rather than accumulated: cognet_1b_latest.pt (overwritten at every save interval), cognet_1b_best.pt (overwritten only when a new best loss is achieved), and cognet_1b_final.pt (written once when training completes). This approach avoids unbounded disk usage from accumulating step-numbered checkpoints. Checkpoints are saved atomically using a write-to-temp-then-rename strategy to prevent corruption from interrupted writes. Optional asynchronous checkpointing offloads the save operation to a background thread, eliminating the training pause that would otherwise occur during checkpoint serialization.

5. Comparison with Transformer Architectures

Aspect	Transformer (GPT-style)	CogNet-1B
Attention mechanism	Self-attention over all positions	SDPA over fixed memory slots
Per-layer complexity	$O(n^2)$	$O(n(S+d)d)$
Context aggregation	Full pairwise attention	Hierarchical memory (3 tiers)
Positional encoding	Learned / RoPE / ALiBi	RoPE (in memory attention)

Aspect	Transformer (GPT-style)	CogNet-1B
Normalization	LayerNorm / RMSNorm	RMSNorm
Feed-forward	MLP / SwiGLU	Fused SwiGLU
Sequence length scaling	Quadratic cost increase	Linear cost increase
Memory for activation	$O(n^2)$ for attention matrix	$O(n)$ for memory slots
Max context window	Limited by $O(n^2)$ cost	Scales linearly with n
Vocabulary	32k-128k BPE	136 character-level

The fundamental architectural difference is in how information flows across sequence positions. Transformers compute attention between all pairs of tokens, yielding a dense interaction graph that captures arbitrary dependencies but at quadratic cost. CogNet instead routes information through a fixed-capacity hierarchical memory system. The memory slots serve as a compressed, learned representation of the sequence context, and their fixed size ensures that the computational cost scales linearly with sequence length regardless of how long the input becomes.

The character-level tokenizer with only 136 tokens is another distinguishing choice. While BPE tokenizers with 32k-128k vocabularies produce shorter sequences for the same text, they also introduce subword boundaries that may not align with meaningful linguistic units. The character-level approach ensures perfect alignment with text while relying on CogNet's $O(n)$ scaling to handle the longer sequences that result from character-level tokenization.

6. Deployment and Reproducibility

6.1 One-Command Launch

The entire CogNet-1B training pipeline can be launched from any SSH terminal with a single command. The `start.sh` script handles repository cloning, dependency installation, GPU detection, and training launch with all optimizations enabled by default. For multi-GPU systems, it automatically configures FSDP via `torchrun`.

```
curl -sL https://huggingface.co/thefinalboss/CogNet-1B/resolve/main/start.sh |
HF_TOKEN=xxx bash
```

6.2 Repository Structure

File	Purpose
<code>cognet_1b_optimized.py</code>	Optimized model architecture (RMSNorm, RoPE, SDPA, Fused SwiGLU)
<code>train_ultra.py</code>	Complete training pipeline (data A-E + benchmark + training)
<code>run.py</code>	Python launcher with auto GPU detection and dependency management
<code>start.sh</code>	SSH one-command launcher for remote GPU instances

File	Purpose
infer_optimized.py	Inference engine (generate, analyze, benchmark)
prepare_data.py	Standalone data preparation script
requirements.txt	Python dependencies
tokenizer_v3.json	Character tokenizer vocabulary
data/	AICL datasets and CogNet training data
checkpoints/	Pre-trained model weights

6.3 Hardware Requirements

For training the 1B parameter model with BF16 mixed precision and gradient checkpointing enabled, a minimum of 8 GB of GPU VRAM is required. 12 GB or more is recommended for comfortable headroom. The model has been tested and verified on NVIDIA A100, H100, RTX 4090, and RTX 3090 GPUs. On CPU, the model functions correctly but training speed is substantially lower. The real benchmark at training startup provides accurate throughput measurements for whatever hardware configuration is used.

7. Future Work

Several directions for future development are being explored. First, scaling the architecture to larger parameter counts (3B, 7B, 13B) will test whether the $O(n)$ complexity advantage persists and amplifies at scale. Second, expanding the memory slot counts and introducing dynamic slot allocation could improve the model's ability to capture long-range dependencies without sacrificing the linear complexity guarantee. Third, replacing the character-level tokenizer with a BPE tokenizer while maintaining the cognitive routing architecture could combine the benefits of subword tokenization with $O(n)$ per-layer scaling.

Additionally, integrating mixture-of-experts (MoE) routing with the cognitive channel mechanism could enable conditional computation where different channels activate for different types of input, further improving the efficiency-to-quality trade-off. Finally, comprehensive evaluation on standard benchmarks (MMLU, HellaSwag, ARC, etc.) will provide direct quality comparisons with transformer-based models of similar parameter counts.

8. References

- [1] Vaswani, A., et al. (2017). "Attention Is All You Need." NeurIPS 2017.
- [2] Shazeer, N. (2020). "GLU Variants Improve Transformer." arXiv:2002.05202.
- [3] Dao, T., et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." NeurIPS 2022.
- [4] Su, J., et al. (2024). "RoFormer: Enhanced Transformer with Rotary Position Embedding." Neurocomputing.

- [5] Zhang, B., & Sennrich, R. (2019). "Root Mean Square Layer Normalization." NeurIPS 2019.
- [6] AFKmoney. "CogNet: Non-Transformer LLM with Cognitive Routing." github.com/AFKmoney/CogNet.
- [7] AFKmoney. "AICL: AI Component Language." github.com/AFKmoney/AICL.
- [8] AFKmoney. "CogNet-1B Pre-trained Model." huggingface.co/thefinalboss/CogNet-1B.